

Relatório de encerramento de temporada — manual

SabatinaQuest — gerado de docs/relatorio-temporada.md

Este documento é o "para sempre" do que o encerramento de **Provado pelo Fogo** (setembro/2026) fez uma vez só, na mão, fora do app. Da próxima vez, o professor extrai o relatório de dentro do próprio app (painel do professor → aba

 **Temporada** → **Baixar relatório (.json)**) e cola no prompt lá embaixo —

sem precisar reconstruir a metodologia do zero, e sem depender de exportação manual do Firestore.

O código que gera esse JSON é `src/relatorioTemporada.ts`, com testes em `tests/relatorioTemporada/`. Qualquer mudança de regra deve mexer nos três lugares juntos: o código, os testes e este documento — os três descrevem a mesma coisa e não podem divergir.

As regras, uma por uma

1. "Dia certo" — quando um estudo conta como feito na data da lição

- **Fonte boa:** `history[dia].emISO`, a data real do estudo, gravada pelo app a partir de **13/09/2026**. Antes dessa data, o campo não existe.
- **Sem `emISO` (dados antigos):** estimado pelo XP do dia. O app multiplica a nota da leitura por 100% no dia certo, 90% na mesma semana e 75% depois (`getRecencyMult` em `src/utils.ts`). Sabendo os **acertos** do dia (0 a 4) e o **XP final**, dá para testar as 3 hipóteses e ver quantas batem.
- **Ambíguo** (mais de uma hipótese bate): desempata pela ****mediana de XP por acerto**** daquele aluno, calculada só sobre os dias que JÁ foram confirmados como "no dia certo" — nunca sobre um palpite. Quem responde tipicamente rápido/devagar puxa o desempate para o lado consistente com o próprio histórico.
- **Nunca conta um dia anterior à criação da conta** do aluno — um doc antigo sem carimbo de turma/conta não pode virar "sequência" antes de a pessoa existir no sistema.

Testado contra os ~250 estudos que já tinham `emISO` real na temporada passada: o método não confundiu nenhum estudo atrasado com um feito no dia certo (ver `tests/relatorioTemporada/relatorioTemporada.test.ts`).

2. Horário do estudo (novo a partir desta temporada)

Desde a atualização que trouxe este relatório, cada dia concluído também grava `history[dia].emHora` (formato `HH:mm`, hora local de quem estudou) ao lado de `emISO`. **Isso não decide nada hoje** — a régua de "dia certo" continua sendo só a data — mas evita ter que fazer outro backfill quando a liderança quiser responder "que horas a turma estuda" (manhã, tarde, noite) num encerramento futuro. Temporadas anteriores a essa mudança não têm esse campo; trate a ausência dele como "sem dado", nunca como "estudou à meia-noite".

3. Maior ofensiva real

Dias **seguidos no calendário**, cada um estudado **no dia certo** (regra 1).

Diferente do 🔥 que o app mostra ao vivo para o aluno (que conta qualquer dia estudado, mesmo atrasado) — aqui um dia atrasado ou adiantado quebra a sequência. Ver `ofensivaReal()`.

4. Melhor semana

A semana com mais estudos feitos no dia certo (regra 1); desempate pelo **percentual de acertos** daquela semana. Ver `melhorSemana()`.

5. Dia mais estudado

A data de calendário com mais alunos estudando **no dia certo** (regra 1) — não conta quem colocou a lição daquele dia em dia mais tarde. Ver `diaMaisEstudado()`.

6. Maratonistas (esforço para colocar em dia)

Quem colocou **mais lições atrasadas em dia num único dia real**. Só entram lições com `emISO` diferente da data da própria lição — e só a partir de 13/09/2026, porque antes disso o app não gravava a data real do estudo e uma maratona anterior não tem como ser provada (vira menção honrosa manual, não dado). Ver `maratonistas()`.

7. Liderança pelo exemplo

Admins e professores, somando os dias estudados em **todas as trilhas** (um professor pode estudar teen e adult ao mesmo tempo). Importante: o colapso de duplicata de doc é por `(userId, semana, trilha)`, não só `(userId, semana)` — colapsar sem a trilha faria duas trilhas da mesma pessoa na mesma semana disputarem a mesma vaga e uma delas sumir da soma. Ver `liderancaPeloExemplo()`.

Quem entra na conta de liderança de uma turma específica: qualquer admin do sistema, mais quem está em `turmas/{id}.professores` — não todo professor do sistema, senão o professor de uma turma herdaria os dias de quem conduz outra.

8. Clube da Temporada / sorteio final

Quem completou **todas** as lições liberadas da temporada, ****mesmo que atrasado**** — reusa a mesma regra do sorteio (`participantesDaTemporada` com `regra: 'temporada-tudo'`, em `src/sorteio.ts`), de propósito: o clube que aparece no encerramento e a urna do sorteio são o MESMO grupo. O top 3 da classificação geral também concorre no sorteio — não é excluído por já ter pódio.

9. Tickets: sorteio semanal e sorteio da temporada (desde 27/09/2026)

- **Sorteio semanal:** só entra quem fez os ****7 dias da semana, cada um no dia certo**** (regra 1). Um dia atrasado, ou faltando, e a pessoa fica fora do sorteio daquela semana. Em código: `semanaTodaNoDia()` em `src/relatorioTemporada.ts`, aplicado pelo Sorteador na regra `semana-completa`.

- **Sorteio da temporada:** continua aceitando atraso (regra 8, sem mudança). Na regra "bilhete por semana", cada semana completa vale 1 ticket — quem juntou mais tickets tem mais chance; na regra "estudou tudo", 1 ticket para quem fez todos os dias liberados.
- O id `semana-completa` foi mantido (em vez de criar um novo) porque `firestore.rules` valida a lista de regras da coleção `sorteios`. Registros semanais feitos antes de 27/09/2026 seguiram a regra antiga, que aceitava atraso — o registro de ganhadores entrou no ar em 26/09/2026, então são pouquíssimos.

O JSON do relatório

A tela do professor gera exatamente a saída de `montarResumoTemporada()` :

```
```jsonc
{
 "turma": "Adolescentes — ASA NORTE",
 "trimestre": "Provado pelo Fogo",
 "geradoEm": "2026-09-27",
 "totais": { "alunos": 14, "dias": 946, "xp": 383349, "semanasCompletas": 131 },
 "ranking": [{ "userId": "...", "nome": "Biazadas", "dias": 91, "xp": 41769, "semanasCompletas": 13 }, "..."],
 "ofensivaReal": [{ "userId": "...", "nome": "Ana Luiza", "dias": 84, "inicio": "2026-07-04", "fim": "2026-09-25" }, "..."],
 "melhorSemana": [{ "week": "2026-W38", "estudosNoDia": 64, "alunos": 12, "pctAcertos": 91.8 }, "..."],
 "diaMaisEstudado": [{ "data": "2026-09-23", "alunos": 11 }, "..."],
 "maratonistas": [{ "userId": "...", "nome": "Davi Kloss", "maxNumDia": 38, "dataMax": "2026-09-19", "totalRecuperado": 46 }, "..."],
 "liderancaPeloExemplo": [{ "nome": "Sioneide Almeida", "dias": 91 }, "..."],
 "clubeDaTemporada": [{ "nome": "Biazadas", "xp": 41769, "dias": 91 }, "..."]
}
```

## Prompt para gerar a próxima apresentação de encerramento

Copie e cole isto num assistente (Claude Code ou outro), colando junto o

arquivo `.json` baixado da aba  Temporada:

Anexo o relatório de encerramento da temporada [NOME DA TEMPORADA], gerado

pelo app (relatorio-temporada-\*.json). Ele já aplica as regras descritas em

`docs/relatorio-temporada.md` do repositório App-Adole- (dia certo, ofensiva

real, melhor semana, dia mais estudado, maratonistas, liderança pelo

exemplo, clube da temporada). Monte um roteiro de encerramento para

adolescentes com: abertura, números da temporada, dia mais estudado, melhor

semana, liderança pelo exemplo, maratonistas, chama acesa (ofensiva real),

clube da temporada, revelação do top 3 em formato de mistério → revelação

(do 3º para o 1º lugar, uma pista de cada vez), resumo de todos os alunos,

sorteio final (regra: clube da temporada inteiro concorre) e virada para a

próxima temporada. Sugira também dicas de apresentação (deixas de música,

silêncio antes da revelação, participação da plateia) e gere o PPT com

transições.

---

## Onde isso mora no sistema

- **Código:** `src/relatorioTemporada.ts` (funções puras, sem I/O) + `src/utils.ts` (`agoraLocalHora`) + `src/App.tsx` (grava `emHora` ao concluir um dia).
- **Testes:** `tests/relatorioTemporada/relatorioTemporada.test.ts`, dentro de `npm run test:unit`.
- **Tela:** painel do professor → aba  **Temporada** (`RelatorioTemporadaAba` em `src/components.tsx`), disponível para qualquer professor na turma que conduz — sem precisar de acesso de admin nem de exportar nada do Firestore na mão.
- **Segurança:** nenhuma regra nova no Firestore foi necessária. `progress/*` já é legível por qualquer usuário autenticado (para o ranking), e `history` é validado só como `map` — o campo `emHora` viaja livre dentro dele.